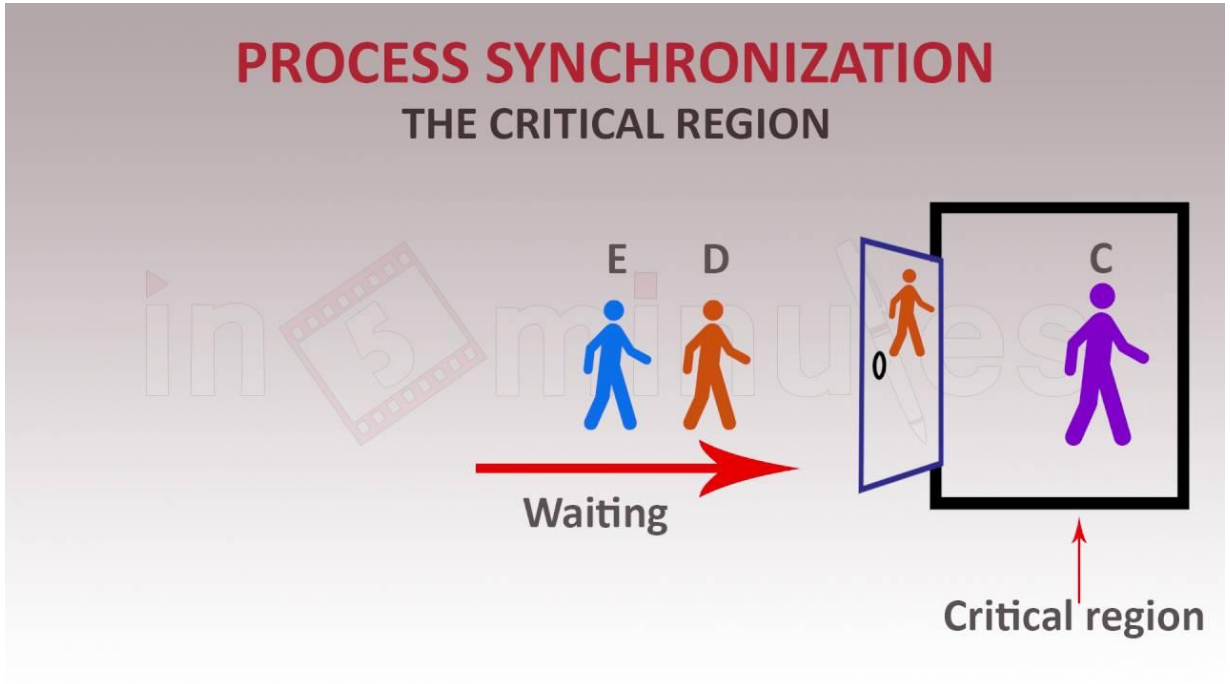
İşletim Sistemleri
(Operating Systems)



PROCESS SENKRONİZASYONU

(Process Synchronization)

Process Senkronizasyonu

- Senkronizasyon temelinde, process'ler iki ayrı sınıfta ele alınır:

- Bağımsız Process (Independent Process): Bir process'in uygulanması, diğer process'lerin yürütülmesini etkilemez.

- Birlikte çalışan Process'ler (Cooperative Processes) : Bir process'in uygulanması, diğer process'lerin yürütülmesini etkiler.

Process Senkronizasyonu

- Birlikte çalışan (Cooperative) process'ler diğer process'leri etkilerler veya diğer process'lerden etkilenirler.
- Bu etkileşme, process'lerin işletimleri sırasında kimi kaynakları ortak kullanmalarından kaynaklanır.
- Process senkronizasyon problemi, Birlikte çalışan Process'lerin yürütümünde ortaya çıkmaktadır çünkü kaynaklar bu yaklaşımda paylaşılmaktadır.
- Paylaşılan kaynak ortak kaynak olarak adlandırılır.

Process Senkronizasyonu

- Senkronizasyon gereksinimi yalnızca çok işlemcili sistemlerinde ortaya çıkmaz.
- Eşzamanlı process yürütümünün olduğu tek işlemcili sistemlerde de gereklidir.
- Process'ler bazen başka bir process'in ürettiği sonuca ihtiyaç duyabilir.
- Bazen de process'ler ortak bellek alanına ya da kaynağa erişmeleri gerekebilir.
- Paylaşılan kaynaklar : değişken, dosya, veri tabanı, ana bellek buffer alanı, donanım aygıtı gibi….
- Bu gibi durumlarda process'ler arasında bir senkronizasyon gerekir.

Process Senkronizasyonu

- Bilgisayar sistem kaynakları :
- a) Bölüşülür kaynaklar (preemptive resources)
- b) Bölüşülmez kaynaklar (non-preemptive resources)

Bölüşülür kaynaklar (preemptive resources)

- Bölüşülür kaynak, bir process tarafından kullanımı tümüyle tamamlanmadan diğer process'ler tarafından da kullanılabilen kaynaktır.
- CPU, ana bellek, sanal bellek (virtual memory) gibi donanımsal kaynaklar bölüşülür kaynaklardır.
- CPU, birden çok process tarafından sırayla paylaşılabilen; bir process'in bu birimle ilgili kullanımı sonlanmadan bir diğer process'in kullanımına da sunulabilen bir kaynaktır.
- Bellek, birden fazla process tarafından paylaşılabilecek şekilde kullanılabilir. Bellek bölgeleri, işletim sistemi tarafından yönetilebilir ve başka bir process'e verilebilir.
- Birden çok process'in, aynı anda ana bellekte saklanıyor olması process'lerin yürütülmesini etkilemez.

Bölüşülmez kaynaklar (non-preemptive resources)

- Bölüşülmez kaynaklar, bir işlemin tamamen serbest bırakmadan başka bir process'e verilemeyen kaynaklardır. Bir process, bu tür bir kaynak üzerinde çalışırken, kaynak tamamen serbest bırakılana kadar başka bir process'e verilemez.
- Bölüşülmez kaynak, bir process tarafından kullanılmaya başlandığında, bu kullanım tümüyle sonlanana değin diğer process'lerin kullanımına sunulamaz.
- **Diskler (Disks):** Bir işlem, disk üzerinde bir I/O işlemi yaparken, disk tamamen serbest bırakılana kadar başka bir işleme verilemez.
- **Yazıcı (Printer):** Bir yazıcı, bir işlem tarafından kullanılmakta ise, yazıcı tamamen serbest bırakılana kadar başka bir işleme verilemez.
- **CD-ROM:** CD-ROM gibi ortak kullanıma açık olmayan cihazlar, bir işlem tarafından tamamen kullanılır ve başka bir işleme verilemez.

Process Senkronizasyonu

- Paylaşılamaz kaynaklar için senkronizasyon genellikle gerekli değildir.
- Senkronizasyonun temel amacı, paylaşılan kaynaklara (ör. bellek bölgeleri, dosyalar, cihazlar vb.) eş zamanlı erişimi kontrol etmek ve veri bütünlüğünü sağlamaktır.
- **Paylaşılamaz kaynaklar ise, aynı anda yalnızca bir process tarafından kullanılabilen veya erişilebilen kaynaklardır. Bu durumda, senkronizasyonun amacı zaten sağlanmış olur.**

Process Senkronizasyonu

- Birlikte çalışan process'ler tarafından paylaşılan değişken, dosya, veri tabanı, ana bellek buffer alanı, donanım aygıtı gibi kaynakların bölüşülür olup olmama özellikleri bu kaynaklar üzerinde yapılan işlemin türüne de bağlıdır.

- **Okuma İşlemi:**
- **Paylaşılabilirlik:** Okuma işlemi için kaynaklar genellikle bölüşülür özellik gösterir. Yani, birden fazla process aynı anda bir kaynağı okuyabilir.
- **Senkronizasyon Gereksinimi:** Eğer tüm process'ler sadece okuma işlemi yapıyorsa, bu durumda herhangi bir senkronizasyon mekanizmasına gerek yoktur. Çünkü okuma işlemi kaynağın durumunu değiştirmez ve process'ler birbirlerini etkilemez.

Process Senkronizasyonu

- **Yazma İşlemi:**
- Bir process paylaşılan bir kaynağa yazma işlemi yaparken, diğer process'lerin aynı kaynağa erişimi kısıtlanmalıdır.
- Yazma işlemi kaynağın durumunu değiştirebileceği için, bu tür erişimlerin düzenlenmesi gerekir.
- Birlikte çalışan process'lerden bazıları paylaşılan kaynağa yazma amacıyla erişiyor ve bazı process'ler de paylaşılan kaynağı okuyorlarsa bu erişimin düzenlenmesi gerekir.

Process Senkronizasyonu

- Process Senkronizasyonu, birden fazla process’in paylaşılan kaynaklara (örneğin, bellek, dosya, veritabanı, donanım aygıtları) erişimini düzenleyerek, bu kaynaklar üzerinde tutarlı ve güvenli bir şekilde işlem yapmalarını sağlayan bir mekanizmadır.
- Process senkronizasyonu, process’lerin birbirlerini beklemelerini veya belirli bir sırayla çalışmalarını sağlar, böylece veri tutarlılığı ve sistem bütünlüğü korunur.

Process Senkronizasyonu Problemleri

- **Yarış Durumu (Race Condition):** İki veya daha fazla process aynı kaynağa eriştiğinde, işlem sırası belirsiz olabilir ve bu da tutarsız sonuçlara neden olabilir.
- **Kayıp Güncelleme (Lost Update):** Bir process'in yaptığı değişiklikler, diğer process'ler tarafından ezilebilir.
- **Deadlock (Kilitlenme):** İki veya daha fazla process birbirinin kaynaklarını beklerken sonsuza kadar bekleyebilir.
- **Starvation (Açlık):** Bir process, kaynaklara erişim hakkını sürekli olarak alamayabilir.

Process Senkronizasyonunun Amacı:

- **Veri Tutarlılığını Sağlamak:** Birden fazla process aynı kaynağa eriştiğinde, bir process'in yaptığı değişikliklerin diğer process'ler tarafından ezilmesini (kayıp güncelleme) önlemek.
- **Yarış Durumlarını (Race Conditions) Engellemek:** Process'lerin kaynaklara erişim sırasının belirsiz olmasından kaynaklanan tutarsızlıkları önlemek.
- **Deadlock (Kilitlenme) ve Starvation (Açlık) Durumlarını Önlemek:** Process'lerin kaynaklara erişim sırasında sonsuza kadar beklememesi veya kaynaklardan mahrum kalmaması için düzenleme yapmak.
- **Process Sıralamasını Kontrol Etmek:** Process'lerin belirli bir sırayla çalışmasını sağlamak (örneğin, bir process diğerinin tamamlanmasını beklemelidir).

Process Senkronizasyonu

- Kritik bölge (critical section), bir programda birden fazla process veya thread tarafından paylaşılan bir kaynağa (örneğin, bellek, dosya, veritabanı, donanım aygıtı) erişen ve bu kaynak üzerinde işlem yapan kod kesimleridir.
- Kritik bölgeler, paylaşılan kaynaklara erişimi düzenlemek ve veri tutarlılığını sağlamak için özel bir öneme sahiptir.
- **Paylaşılan kaynağa aynı anda en çok bir process'in erişebilmesi kaynağa erişim yapan process'lerden en çok birinin kritik bölge içinde bulunmasının sağlanmasıyla gerçekleşebilir.**
- Bir process'in kaynaklara erişen kod bölümü (kritik bölge) işlemi tamamlanmadan başka bir process kritik bölge kodlarını yürütmemelidir.

Yarış Durumu (Race condition)

- Yarış durumu (race condition) multiprocess ve multithread programlamanın en büyük sorunlarından biridir.

- Process'lerin kritik kaynağa erişmek için yarış içinde oldukları durumdur.

- Yarış Durumu (Race Condition), birden fazla process veya thread'in aynı anda paylaşılan bir kaynağa (örneğin, bellek, dosya, veritabanı) erişmeye çalıştığı ve bu erişimin sırasının belirsiz olduğu durumlarda ortaya çıkan bir senkronizasyon problemidir.
- Bu durum beklenmeyen, tutarsız sonuçlara neden olabilir.

Yarış Durumu (Race condition)

- Yarış durumu, aşağıdaki koşullarda ortaya çıkar:
- **Paylaşılan Kaynak:** İki veya daha fazla process veya thread aynı kaynağa erişir.
- **Eşzamanlı Erişim:** Process'ler veya thread'ler aynı anda kaynağa erişmeye çalışır.
- **Senkronizasyon Eksikliği:** Kaynağa erişim düzenlenmemiştir.

- **Bu durumda, process'lerin veya thread'lerin kaynağa erişim sırası belirsizdir ve bu da tutarsız sonuçlara yol açar.**

Yarış Durumu (Race condition)

Time	You	Your roomate
3:00	Arrive home	
3:05	Check fridge → no milk	
3:10	Leave for grocery	
3:15		Arrive home
3:20	Buy milk	Check fridge → no milk
3:25	Arrive home, milk in fridge	Leave for grocery
3:30		
3:35		Buy milk
3:40		Arrive home, milk in fridge!

Kaynakları paylaşan kooperatif process'lerin davranışlarını nasıl kontrol edersiniz?

Yarış Durumu (Race condition)

int Count = 10;

Process 1

Count++;

Process 2

Count--;

Count=10 iken;

Count++ ve Count-- komutlarını içeren process'lerin eşzamanlı çalıştığını düşünelim.

Tek tek çalıştırıldığında, yandaki kod blokları doğru şekilde çalışır.

Fakat eşzamanlı doğru olarak çalışmayabilirler.

Bir process'in çalışması sırasında hangi zamanlarda kesintiye uğrayacağı önceden kestirilemez.

Process'lerin birlikte çalışmaları sırasında, bu kod bloklarının yürütülmesi sonrasında **Count** değeri birkaç farklı değer alabilir.

Bu yüzden sonuçta veri tutarsızlığı olayı yaşanabilir.

Yarış Durumu (Race condition)

Count++ için makine komutları aşağıdaki gibi olabilir.

LOAD Reg, Count

ADD #1

STORE Reg, Count

Count-- için makine komutları :

LOAD Reg, Count

SUB #1

STORE Reg, Count

Count = ? 9, 10 , 11?

Yarış Durumu (Race condition)

Context switching herhangi bir zamanda, hatta bir kod satırının ortasında bile oluşabilir.
Count++ ve Count-- için sıralı zaman aralıklarında yapılan mikroişlemler aşağıdaki gibi olabilir.
Bu deyimler farklı zaman aralıklarında çalışmış olsalardı Count=10 olacaktı.

Process 1			Process 2		
Instruction	**Register**	**Memory**	**Instruction**	**Register**	**Memory**
LOAD	10	10			
			LOAD	10	10
			SUB	9	10
ADD	11	10			
STORE	11	**11**			
			STORE	9	**9**

Yukarıdaki sırada memory değeri 9 olarak görülür, ancak **gerçekte memory değeri 10 olmalıdır**.
Son iki işlem yer değiştirirse bu sefer memory değeri 11 olarak görülecektir.

Yarış Durumu (Race condition)

Count++ ve Count-- için sıralı zaman aralıklarında yapılan mikroişlemler aşağıdaki gibi de olabilir.
Bu deyimler farklı zaman aralıklarında çalışmış olsalardı Count=10 olacaktı.

Process 1			Process 2		
Instruction	**Register**	**Memory**	**Instruction**	**Register**	**Memory**
LOAD	10	10			
ADD	11	10			
			LOAD	10	10
			SUB	9	10
			STORE	9	**9**
STORE	11	**11**			

Yukarıdaki sırada memory değeri 11 olarak görülür, ancak **gerçekte memory değeri 10 olmalıdır**.

Yarış Durumu (Race condition)

- Örnek: a=b korunacak başta a=1, b=1

P1:
```
a=a+1;
b=b+1;
```

P2:
```
b=2*b;
a=2*a;
```

Sıralı çalışırsa sonuçta
a=4 ve b=4 √

```
a=a+1;
b=2*b;

b=b+1;
a=2*a;
```

Bu sırayla çalışırsa sonuçta
a=4 ve b=3 X

Yarış Durumu (Race condition)

- Örneğin A ve B isimli iki işlemin “Hesap” isimli orak veri alanı üzerinde işlem yaptığını varsayalım.
- Başlangıçta Hesap değişkeninde 1000$ olduğunu varsayalım.
- A işlemi, banka hesabına 100$ para yatırılması,
- B işlemi ise aynı hesaptan 200$ çekilmesini gerçekleştirmektedir ve iki işlem de hemen hemen aynı zamanda başlatılmıştır.

Yarış Durumu (Race condition)

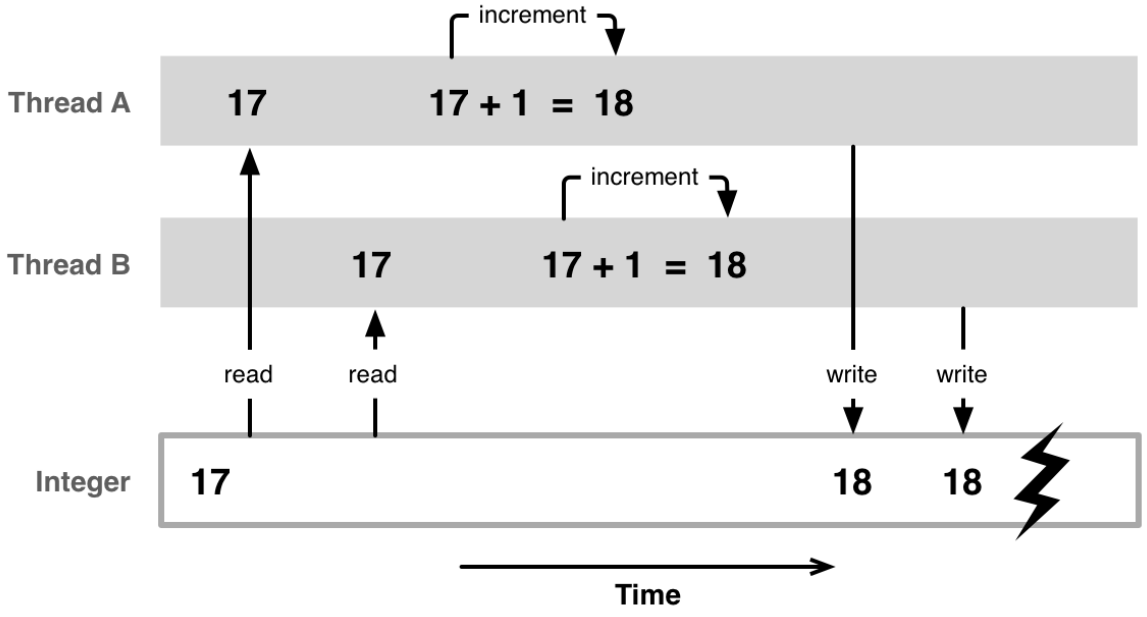
- **İşlem A**
- A1. Hesap değerini oku ve yerel YerelHesapA değişkenine ata.
- A2. YerelHesapA ← YerelHesapA +100
- A3. YerelHesapA değişkeni içindeki değeri Hesap değeri olarak yaz.
- **İşlem B**
- B1. Hesap değerini oku ve yerel YerelHesapB değişkenine ata.
- B2. YerelHesapB ← YerelHesapB - 200
- B3. YerelHesapB değişkeni içindeki değeri Hesap değeri olarak yaz.

Yarış Durumu (Race condition)

- Bu durumda Hesap değişkeni üç farklı son değere sahip olabilir.
- **I.Durum**
- CPU önce İşlem A'yı ardından İşlem B'yi veya önce İşlem B'yi ardından İşlem A'yı çalıştırırsa Hesap değişkeninde 900$ bulunmaktadır. Bu strateji işlemi bütün olarak çalıştırmaktadır.
- **II.Durum**
- Bu çalışma sırasında A ve B aynı değerleri okuyup yerel kopyalarında değişiklik yapıyorlar ve yerel kopyasını son yazan (ki bu durumda A) Hesap değişkeninin aldığı son değeri belirliyor (1100).
- **III.Durum**
- Son yazanın Hesap değişkenini belirlemesi burada da İşlem B tarafından gerçekleştiriliyor (800).

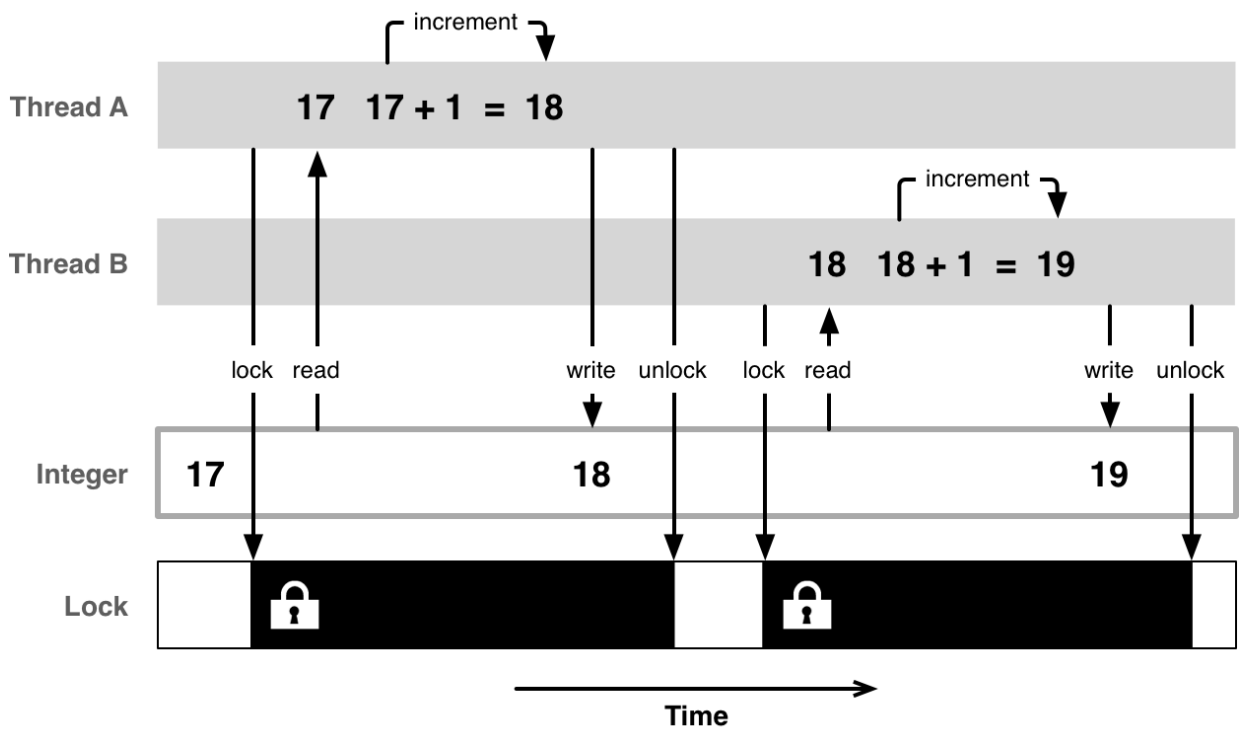
Yarış Durumu (Race condition)

- İki thread aynı anda bir işlemi yapmaya çalışırsa oluşabilecek durumu düşünelim.
- Örneğin, thread A ve thred B, her ikisi de bir counter değerini bellekten okusun; counter değeri başlangıçta 17 olsun.
- Ardından thread A counter değerini bir arttırır ve 18'i belleğe yazar. Aynı zamanda, thread B counter değerini bir arttırır ve 18'i thread A dan hemen sonra belleğe yazar.
- Bu noktada, veri bozulmuştur. çünkü counter değeri 17'den iki kez arttıktan sonra hala 18 değerini tutmaktadır.



Yarış Durumu (Race condition)

- Bir process/thread paylaşılan bir kaynağa erişim başlamadan önce, başka bir process/thread çalışmasının paylaşılan kaynak üzerinde bitmiş olduğundan emin olmalıdır.



Karşılıklı dışlama (mutual exclusion)

- **Karşılıklı Dışlama (Mutual Exclusion)**, birden fazla process veya thread'in aynı anda paylaşılan bir kaynağa (örneğin, bellek, dosya, veritabanı) erişmesini engelleyen bir senkronizasyon mekanizmasıdır.
- Bu mekanizma, aynı anda sadece bir process veya thread'in kritik bir kaynağa erişmesine izin verir ve diğerlerinin erişimini engeller.
- Karşılıklı dışlama, veri tutarlılığını ve sistem bütünlüğünü korumak için kullanılır.
- Karşılıklı dışlama process'lerin kritik bölgelerine giriş ve çıkışlarında alınan önlemler yoluyla gerçekleştirilir.

Karşılıklı dışlama (mutual exclusion)

- **Karşılıklı Dışlamanın Amacı:**
- **Veri Tutarlılığını Sağlamak:** Birden fazla process veya thread aynı kaynağa eriştiğinde, bir process'in yaptığı değişikliklerin diğer process'ler tarafından ezilmesini (kayıp güncelleme) önlemek.
- **Yarış Durumlarını (Race Conditions) Engellemek:** Process'lerin veya thread'lerin kaynaklara erişim sırasının belirsiz olmasından kaynaklanan tutarsızlıkları önlemek.
- **Kritik Bölgeleri Koruma Altına Almak:** Kritik bölgeye (critical section) aynı anda sadece bir process veya thread girebilmelidir.

Karşılıklı dışlama (mutual exclusion)

- **Karşılıklı dışlama, aşağıdaki adımlarla çalışır:**
- **Kritik Bölgeye Giriş:** Bir process veya thread, kritik bölgeye girmeden önce bir kilidi (lock) alır. Eğer kilidi alamazsa (başka bir process veya thread kilidi almışsa), bekler.
- **Kritik Bölgede İşlem Yapma:** Kilidi alan process veya thread, kritik bölgede işlem yapar.
- **Kritik Bölgeden Çıkış:** İşlem tamamlandıktan sonra, process veya thread kilidi serbest bırakır.
- Daha sonra diğer process veya thread'ler kilidi alabilir ve kritik bölgeye girebilir.

Karşılıklı dışlama (mutual exclusion)

- Karşılıklı Dışlamanın Dezavantajları:

- **Performans Maliyeti:** Kilitleme mekanizmaları, process'ler veya thread'ler arasında ekstra bir yük getirebilir.
- **Deadlock (Kilitlenme):** İki veya daha fazla process veya thread birbirinin kilidini beklerken sonsuza kadar bekleyebilir.
- **Starvation (Açlık):** Bir process veya thread, kilidi sürekli olarak alamayabilir.

Kritik Bölge Problemi
(Critical Section Problem)

- Program kodunun, bölüşülmez kritik kaynaklar üzerinde işlem yapılan kısmı **Kritik Bölge (Critical Section)** olarak adlandırılır.

- Process kritik bölgede ortak değişkenleri değiştiriyor, tablo güncelliyor, dosyaya yazıyor vs. olabilir.

- Kritik bölge, aynı anda yalnızca bir process tarafından erişilebilen bir kod kesimidir.

- Kritik bölge atomik (bölünemez) olarak çalıştırılması gereken kod parçasıdır.

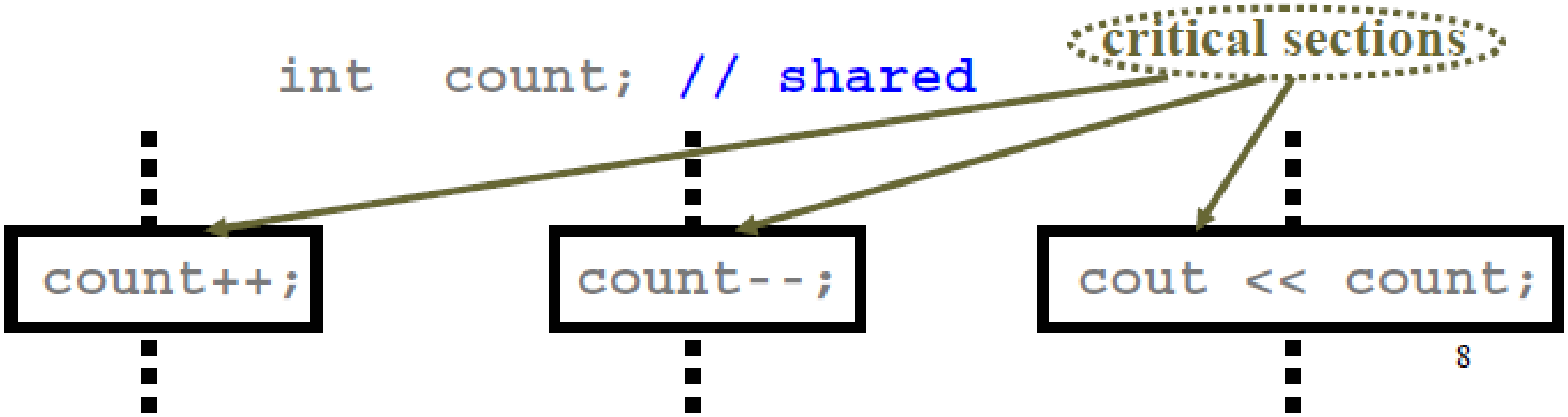
Kritik Bölge Problemi
(Critical Section Problem)

- Paylaşılan veri değişkenlerinin tutarlılığını sağlamak için process'lerin kritik bölgeye erişiminde senkronize edilmeleri gereklidir.

- **Bir process kritik bölge kodunu yürütüyorsa, aynı anda başka bir process kendi kritik bölge kodunu yürütmemelidir.**
- Yani belirli bir anda sadece tek bir process kritik bölgesindeki kodu yürütebilir.

- Her process, kritik kod bölümüne girmek için izin istemek zorundadır.

Kritik Bölge Problemi
(Critical Section Problem)



Kritik bölge (critical section), paylaşılan kaynakları içerir.

Bir process kendi kritik bölgesini yürütürken diğer process'lerin kendi kritik bölgelerini yürütmemeleri zorunludur.

Kritik Bölge Problemi
(Critical Section Problem)

- Her process, ortak değişkenler, tablolar veya dosyalar üzerinde işlem yapan kritik bölgeye sahip olabilir.

- ***Kritik bölge, bir giriş bölümü (entry section) ve bir çıkış bölümü (exit section) arasında bulunur.***
- ***entry section*** giriş izni için kullanılır.
- ***entry section*** kritik bölgeye girmeden önce, atomik olarak kilitler.
- Kritik bölge kilitliyse, kilit açılana kadar diğer process'lerin kritik bölgelerine girmeleri engellenir.
- ***exit section*** kilidi açar. Kritik bölge işlemini serbest bırakır.
- **remainder section** – kalan bölüm

```
do {
    entry section
        critical section
    exit section
        remainder section
} while (TRUE);
```

Kritik Bölge Probleminin Çözümü (Solution to Critical-Section Problem)

- Kritik bölge sorununun çözümü aşağıdaki üç gereksinimi karşılamalıdır:

(1) Karşılıklı Dışlama (Mutual Exclusion)

(2) İlerleme (Progress)

(3) Sınırlı Bekleme (Bounded Waiting)

(1) Karşılıklı Dışlama (Mutual Exclusion)

- Bir process kritik bölge kodunu yürütülüyorsa, başka hiçbir process kritik bölgede kod yürütmemelidir (kod bölümlerini kilitleyebilmek gerekir).
- Aynı zamanda kritik bölge kodlarının da bölünmez şekilde atomik olarak yürütülmesi gerekir.
- Dolaysıyla process'ler arasında karşılıklı dışlamayı sağlamanın temelde iki değişik çözümü bulunmaktadır:

- 1. Kilit mekanizması
- 2. Atomik değişkenler ve kesilmez işlemler

(1) Karşılıklı Dışlama (Mutual Exclusion)

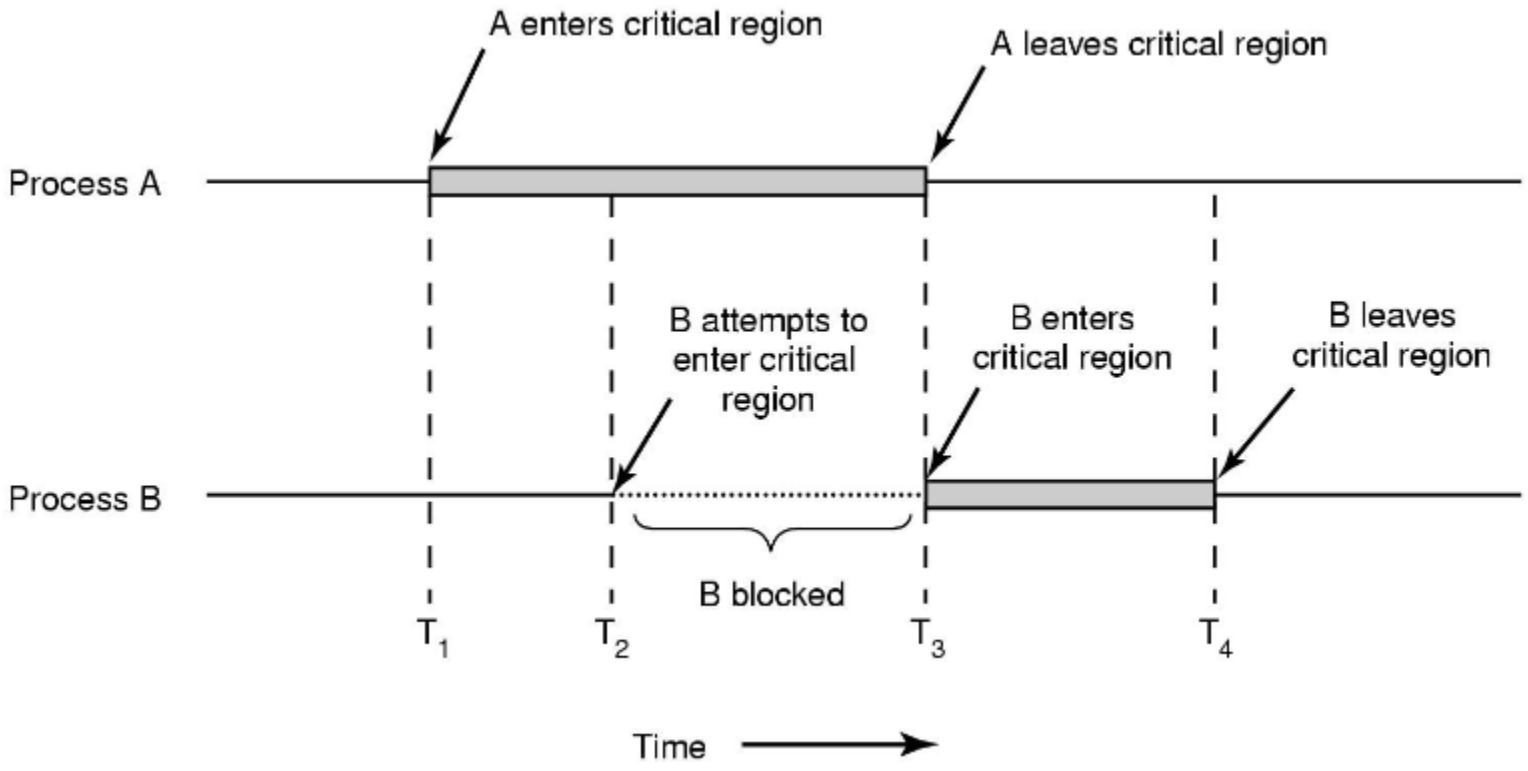
- Paylaşılan kaynaklara iki tür erişim yapılır:
- **Yazma ve Okuma**
- <u>Yazmada karşılıklı dışlama olmalı</u>
- Okuma için karşılıklı dışlama gereksiz

(1) Karşılıklı Dışlama (Mutual Exclusion)

- Karşılıklı dışlama çözümü, çok process'li programlamada yarış durumlarını (race condition) engeller.

- Ancak karşılıklı dışlamanın kullanıldığı çözümde, process'lerin işlemci kullanım başarımı düşer.

(1) Karşılıklı Dışlama (Mutual Exclusion)



Karşılıklı Dışlama (Mutual Exclusion) ile kritik bölge probleminin çözümü

(2) İlerleme (Progress)

- Hiçbir process Kritik bölgede işlem yürütmüyorsa ve kritik bölgesine girmek isteyen başka process'ler mevcutsa, bir tanesinin kritik bölgesine girmesine izin verilir.

- Kritik bölgeye girecek process'lerin seçimi süresiz ertelenemez.

(3) Sınırlı Bekleme (Bounded Waiting) :

Sınırlı bekleme, hiçbir process'in kritik bölgesine girmek için sınırsız süre boyunca beklememesi gerektiği anlamına gelir.

Kritik bölgeye girmek isteyen bir process'in bekleme süresi sınırlandırılmalıdır, bundan sonra kritik bölgesine erişebilmelidir,

yani sistem, sabit bir süre sonra process'in kesinlikle kritik bölgesine erişebileceğini söyleyebilmelidir.

Kritik Bölge Probleminin Olası Çözümleri

- **Donanım Gerektirmeyen Yazılıma Dayalı Çözümler**
- **Yazılım ve Donanıma Dayalı Çözümler**
- **Donanıma Dayalı Çözümler**

Donanım Gerektirmeyen Yazılıma Dayalı Çözümler

Dekker Algoritması

Dekker Algoritması

- Dekker algoritması kritik bölge probleminin çözümü için icat edilmiş ilk algoritmalardandır.
- 1964'te T. J. Dekker tarafından geliştirilmiştir.
- İki process için geliştirilmiş bir metottur.
- Yani ikiden fazla sayıda process varsa uygulanabilirliği yoktur.

Dekker Algoritması

- İlk versiyon
  - Sıralı çalışma. Algoritmada ilerleme (progress), garanti edilmez.
  - Yürütme hızı alt process tarafından belirlenir.
- İkinci versiyon
  - Karşılıklı dışlanmayı (mutual exclusion) garanti etmez.
- Üçüncü versiyon
  - Kilitlenme (dead lock)
- Dördüncü versiyon
  - Karşılıklı nezaket (mutual courtesy)
- Son versiyon
  - Karşılıklı Dışlama (Mutual Exclusion)
  - İlerleme (Progress)
  - Sınırlı Bekleme (Bounded Waiting)
  - Yürütme hızı daha hızlı process tarafından belirlenir,

Dekker Algoritması

- **Dekker Algoritmasının Birinci Versiyonu : Sıralı Çalışma (Strict Alternation)**
- Başlangıçta P1 ve P2 olmak üzere iki process mevcuttur ve kritik bölümlerinin yürütülmesi istenmektedir.
- **turn**, her iki process için ortak bir integer değişkendir. Bu değişken, hangi process'in kritik bölgeye gireceğini belirlemek için kullanılır.

```
int turn=1;
```

```
process P1
while (true) {
  nonCriticalSection_P1;
  while (turn! = 1 ) { }
  criticalSection_P1;
turn = 2;
}
end P1;
```

```
process P2
while (true) {
  nonCriticalSection_P2;
  while (turn!= 2) { }
  criticalSection_P2;
turn = 1;
}
end P2;
```

Mutual exclusion
OK

Progress
X

Bounded waiting
X

Bu yaklaşım karşılıklı dışlamayı (mutual exclusion) sağlar, Ancak process'ler değişmez bir sıra ile çalışmak zorundadır.

Dekker Algoritması

Dekker Algoritmasının Birinci Versiyonu : Sıralı Çalışma (Strict Alternation)

```
int turn=1;
```

```
process P1
while (true) {
 nonCriticalSection_P1;
  while (turn! = 1) { }
  criticalSection_P1;
turn = 2;
}
end P1;
```

```
process P2
while (true) {
 nonCriticalSection_P2;
  while (turn!= 2) { }
  criticalSection_P2;
turn = 1;
}
end P2;
```

- Process ilk olarak **turn** değişkenini sınayacak, şayet **turn** değişkeninin değeri process'in numarası ile eşleşiyorsa process kritik bölgesine girecek aksi taktirde bekleyecektir.
- turn değişkeni 1 olduğunda P1 kritik bölgeye girme şansını yakalar. P1 kritik bölgeyi bitirene kadar turn değeri 1 olarak kalır.
- P1 kritik bölgeyi bitirir ve turn değişkenine 2 atar. P2 kritik bölgeye girme şansını yakalar. P2 kritik bölgeyi tamamlayana kadar turn değeri 2 olarak kalır. P2 kritik bölgeyi bitirir ve turn değişkenine 1 atar.
- Algoritma iki process arasında bir sıra belirler ve her process sırayla kritik bölgeye girebilir.

Dekker Algoritması

- **Dekker Algoritmasının Birinci Versiyonu : Sıralı Çalışma (Strict Alternation)**
- Bu algoritma yalnızca iki process için çalışır.
- Process'ler farklı hızlarda çalışabilecek olmalarına rağmen ancak sıralı şekilde çalışmak zorundadır.
- **Algoritmada ilerleme (progress), garanti edilmez.**
- Bu algoritma ile process'ler kritik bölgelerine "bir o bir bu" diye nitelenebilen değişmez bir sırada erişebilmektedirler. Her process yürütülmesi için diğerine bağımlıdır.
- Bir process kritik bölgesinden çıkışta erişim sırasını diğer process'e bırakmakta, diğer process kritik bölgesine girmediği sürece ilk process bir daha kritik bölgesine girme hakkı kullanamamaktadır.
- Bu da, bir process'in kritik bölgesi dışında olmasına karşın diğer process'lerin kritik bölgelerine girmelerini engelleyen bir durum oluşturmaktadır.

Dekker Algoritması

- **Dekker Algoritmasının Birinci Versiyonu : Sıralı Çalışma (Strict Alternation)**
- Dekker algoritması, iki process'in kritik bölgeye sırayla girmesini sağlamak için bir meşgul bekleme (busy waiting) mekanizması kullanır.
- Dekker algoritmasında, her iki process de kritik bölgeye girmek için beklerken döngü içinde işlemciyi sürekli olarak meşgul ederler ve işlemcinin verimli bir şekilde kullanılmasını engelleyerek, sistem performansını düşürebilir.
- Bekleme, waiting/bloke durumda gerçekleşmediğinden CPU'nun kullanılmamasına sebep olur. Bu da kaynak israfıdır.

Dekker Algoritması

- **Dekker Algoritmasının Son Versiyonu :**

```
int c1=1;
int c2=1;
int turn=1;
```

```
process P1
while (true) {
  nonCriticalSection1;
  c1=0;
  while (c2!=1)
    if (turn==2){
      c1=1;
      while (turn!=1) { }
      c1=0;
    }
  criticalSection1;
  c1=1; turn=2;
}
end P1;
```

```
process P2
while (true) {
  nonCriticalSection2;
  c2=0;
  while (c1!=1)
    if (turn==1){
      c2=1;
      while (turn!=2) { }
      c2=0;
    }
  criticalSection2;
  c2=1; turn=1;
}
end P2;
```

Mutual exclusion,	Progress,	Bounded waiting
OK	**OK**	**OK**

Process’ler arasında paylaşılan üç değişken vardır:

c1: Process 1'in kritik bölgeye girecek durumda olup olmadığını belirten bayrak.

c2: Process 2'in kritik bölgeye girecek durumda olup olmadığını belirten bayrak.

turn: Hangi process’in kritik bölgeye erişme hakkının olduğunu belirten sıra bayrağı.

her iki process’in c1 ve c2 değişkenleri 0 olarak ayarlandığında, turn değişkeni process sırasını gösterir.

Dekker Algoritması

- **Dekker Algoritmasının Son Versiyonu :**
- İki process de birbirlerinin c1 ve c2 bayraklarını ve turn değişkeni ile sırasını kontrol ederek kritik bölgeye erişim için yarışır.

- Bu şekilde, bir process kritik bölge kodunu yürütürken, diğer process bekler ve sırayla kritik bölgeye erişim sağlanır.

Dekker Algoritması

Dekker Algoritmasının Son Versiyonu :

```
process P1
while (true) {
  nonCriticalSection1;    // Kritik olmayan bölge
  c1 = 0;                 // Kritik bölgeye girmek istendiğini belirt
  while (c2!= 1)          // Diğer process'in kritik bölgede olup olmadığını kontrol et
   if (turn == 2) {       // Eğer sıra diğer process'de ise
     c1 = 1;              // Kritik bölgeye girişi iptal et
     while (turn!= 1) { } // Sıranın kendisine gelmesini bekler
     c1 = 0;              // Kritik bölgeye girmek istediğini belirt
   }
  criticalSection1;       // Kritik bölgeye giriş
  c1 = 1;                 // Kritik bölgeden çıkışı belirt
  turn = 2;               // Sırayı diğer process'e ver
}
end P1;
```

P1 process'i İlk olarak, nonCriticalSection1 adı verilen bölgede kritik olmayan işlemleri gerçekleştirir.
Ardından, c1 değişkenini 0 olarak ayarlar ve c2 değişkeni 1 olana kadar bekler.
Eğer turn değişkeni 2 ise, diğer process'in sırası olduğu anlamına gelir.
Bu durumda, c1 değişkenini 1 yapar ve turn değişkeni 1 olana kadar bekler.
Daha sonra tekrar c1 değişkenini 0 yapar.

Bu adımların amacı, process'in kritik bölgeye girmek istediğini ve sıranın kendisine gelmesini beklediğini belirtmektir.

Son olarak, criticalSection1 adı verilen kritik bölümdeki işlemleri gerçekleştirir, c1 değişkenini 1 yapar ve turn değişkenini 2 olarak ayarlar.

Dekker Algoritması

Dekker Algoritmasının Son Versiyonu :

```
process P2
while (true) {
  nonCriticalSection2;          // Kritik olmayan bölge
  c2 = 0;                       // Kritik bölgeye girmek istendiğini belirt
  while (c1 != 1)               // Diğer process'in kritik bölgede olup  olmadığını kontrol et
    if (turn == 1) {            // Eğer sıra diğer process'de ise
      c2 = 1;                   // Kritik bölgeye girişi iptal et
      while (turn != 2) { }     // Sıranın kendisine gelmesini bekler
      c2 = 0;                   // Kritik bölgeye girmek istediğini belirt
    }
  criticalSection2;             // Kritik bölgeye giriş
  c2 = 1;                       // Kritik bölgeden çıkışı belirt
  turn = 1;                     // Sırayı diğer process'e ver
}
end P2;
```

Dekker Algoritması

- Dekker algoritması, iki process'in kritik bölgeye aynı anda erişmesini önlemek için kullanılan bir senkronizasyon algoritmasıdır.
- Dekker Algoritması matematiksel yönden tam doğrudur ve Karşılıklı Dışlama (Mutual Exclusion), İlerleme (Progress) ve Sınırlı Bekleme (Bounded Waiting) problemini çözmüştür.
- Ancak algoritma oldukça karmaşıktır.
- Aynı zamanda bu algoritma en çok iki process'in senkronizasyonu için kullanılabilmekte, ikiden çok process için genelleştirilememektedir.
- Yani ikiden fazla sayıda process varsa uygulanabilirliği yoktur.
- Bu önemli kısıtlamadan dolayı pratikte uygulanamamıştır.

- **Bunun yanında Peterson da kritik bölge problemine yönelik bir algoritma geliştirmiştir.**

Peterson Algoritması

- Peterson'un çözümünün anlaşılması daha kolaydır. Peterson'un orijinal algoritması sadece iki process ile çalışır. Fakat bu algoritma ikiden fazla process için genelleştirilebilir.

```
boolean flag[2]={false, false};
int turn;
```

```
process P1
do {
  flag[0]=true;
  turn=1;
  while (flag[1] && turn = = 1);
  criticalSection1;
  flag[0]=false;
   }
while (true);
end P1;
```

```
process P2
do {
flag[1]=true;
turn=0;
while (flag[0] && turn = = 0);
criticalSection2;
flag[1]=false;
  }
while (true);
end P2;
```

Mutual exclusion, **OK**

Progress, **OK**

Bounded waiting **OK**

Bu yöntemde de bir dizi oluşturulur ve her process için dizide bir eleman tutulur.
Her process dizide kendisi için tutulan değişkeni 1 veya 0 yapar.
Örneğin Process1 kritik bölgeye girdiğinde dizideki değişkenini 1 yapar.
Process2 de diziye girememiş olur.
Process1'in değişkeni 0 olmadan Process2 kendi değişkenini 1 yapamaz.

Peterson Algoritması

```
boolean flag[2]={false, false};
int turn;
```

Bu process kritik bölgeye girmeye hazır

Diğer process kritik bölgeye girmeye hazırsa öncelik ona verilir.

Sıra kendisine gelene kadar bekler

Kritik bölgeden çıkış bildirimi (flag[0] = false)

```
process P1
do {
  flag[0]=true;
  turn=1;
  while (flag[1] && turn = = 1);
  criticalSection1;
  flag[0]=false;
   } while (true);
end P1;
```

```
process P2
do {
flag[1]=true;
turn=0;
while (flag[0] && turn = = 0);
criticalSection2;
flag[1]=false;
  } while (true);
end P2;
```

Peterson Algoritması

```
boolean flag[2]={false, false};
int turn;
```

```
process P1
do {
  flag[0]=true;
  turn=1;
  while (flag[1] && turn = = 1);
  criticalSection1;
  flag[0]=false;
   } while (true);
end P1;
```

```
process P2
do {
flag[1]=true;
turn=0;
while (flag[0] && turn = = 0);
criticalSection2;
flag[1]=false;
  } while (true);
end P2;
```

Eğer turn = 0 ise, kritik bölgeye P1 girecektir.

flag[ ] bitleri ise process'lerin kritik bölgeye girmeye hazır durumunu gösterir.

Eğer flag[0] = true ise, P1 kritik bölgeye girmeye hazırdır.

flag[0] = true VE turn = 0 olunca P1 kritik bölümüne girer.

turn değişkenini iki process'te aynı anda değiştirse bile, son değer alınır ve o process kritik bölgeye girer (mutual exclusion).

Kritik bölgeyi tamamlayan process kritik bölgeye giriş isteğini iptal eder ve diğer process kritik bölgeye girer (progress).

Bir process kritik bölgeye bir kez girdikten sonra sırayı diğerine aktarır (bounded waiting).

Peterson Algoritması

- Peterson‘un orijinal algoritması sadece iki process ile çalışır ve anlaşılması daha kolaydır.
- Bu algoritma ikiden fazla process için genelleştirilebilir (process sayısı sabitlenmiş ve önceden bilinmelidir). Fakat özellikle çok sayıda process için genişletilmesi karmaşık ve anlaşılması zor olabilir.
- Peterson algoritması, process’ler arasında sıra bazlı bir erişim kontrolü sağlar. Bu durum, bazı durumlarda sistem performansını düşürebilir, özellikle kritik bölgeye erişim sıklığı çok yüksekse.

Peterson Algoritması

- Peterson algoritması da doğru bir şekilde kullanılabilir fakat bu metot meşgul beklemeye (busy waiting) yani işlemcinin boşuna kullanılmasına sebep olur.
- Busy Waiting (meşgul bekleme): Peterson algoritması, kritik bölgeye erişim için bekleyen process'lerin meşgul bekleme yapmasını gerektirir. Bu durum, process'lerin sürekli olarak birbirlerini kontrol etmeleri anlamına gelir ve bu da CPU kaynaklarının verimsiz kullanımına yol açabilir.
- Peterson algoritması genellikle eğitim amaçlı olarak kullanılır ve gerçek dünya uygulamalarında daha karmaşık ve etkili senkronizasyon teknikleri tercih edilir.

Kritik Bölge Probleminin Olası Çözümleri

- **Donanıma Dayalı Çözümler**

Donanıma Dayalı Çözümler

- Yazılım tabanlı kritik bölge problemi çözümlerinin (Peterson'un algoritması gibi) modern bilgisayar mimarilerinde çalışmasının garantisi yoktur.

- Özellikle kritik bölge probleminin çözümünde, karşılıklı dışlama kapsamında yazılımsal yaklaşımlarda process'ler, kaynağın serbest olup olmadığını sürekli olarak sınamak suretiyle CPU'yu gereksiz yere meşgul ediyorlardı.

Donanıma Dayalı Çözümler

- Peterson'un algoritması, N process için genelleştirilebilir.
- Fakat,

➢ N sabitlenmiş ve önceden bilinmelidir.

➢ Aynı zamanda, bu algoritmalar çok karmaşıktır ve karşılıklı dışlama mekanizmasının uygulanması zordur!

- Bu yüzden kritik bölge problemini donanım temelli çözmek için bazı yöntemler geliştirilmiştir.

Donanıma Dayalı Çözümler

- Kritik bölge probleminin çözümü için İki çeşit donanıma dayalı senkronizasyon yöntemi vardır:

- **Kesmeleri kapatmak/açmak (Disabling/Enabling interrupts) :**

➢Bu yöntem, çok işlemcili sistemlerde yavaş ve uygulanması zor.

- **Özel ayrıcalıklı makine komutları kullanmak :**

➢Test and set (TS)

➢Swap

➢Compare and Swap (CS)

Kesmeleri kapatmak/açmak (Disabling/Enabling interrupts)

- Donanım temelli çözüm yöntemlerinden ilk akla gelen, kesmelerin (interrupt) kritik bölge kod yürütülmesi esnasında kapatılmasıdır.

- Tek işlemcili sistemlerde, kritik bölge problemi paylaşılan veriler üzerinde process'lerin kesme (interrupt) almadan gerçekleştirilmesiyle çözümlenebilir.

- Böylece paylaşılan kaynaklar üzerinde işlem yapılırken, başka bir değişikliğin yapılması söz konusu olmayacaktır.

Kesmeleri kapatmak/açmak (Disabling/Enabling interrupts)

- Bir process kritik bölgesine girince sistem kesmelere (interrupt) karşı kapatılarak diğer process'lerin ilgili kritik bölgesine girmesi engellenebilir ve kritik bölge işlemi tamamlanınca da tekrar kesmelere açılabilir.
- Böylece process kritik bölge işlemini bitirene kadar CPU'yu kullanır ve başka bir process'in CPU'dan faydalanmasını engeller.

Process 1

```
...
kesmeler_pasif;
toplam=toplam + degisim;
kesmeler_aktif;
...
```

Process 2

```
...
kesmeler_pasif;
toplam=toplam - degisim;
kesmeler_aktif;
...
```

Kesmeleri kapatmak/açmak (Disabling/Enabling interrupts)

Kesmelerin Aktif ve Pasif Yapılması :

Process 1

```
di;
saniye1 = saniye;
dakika1 = dakika;
saat1 = saat;
ei;
```

Process 2

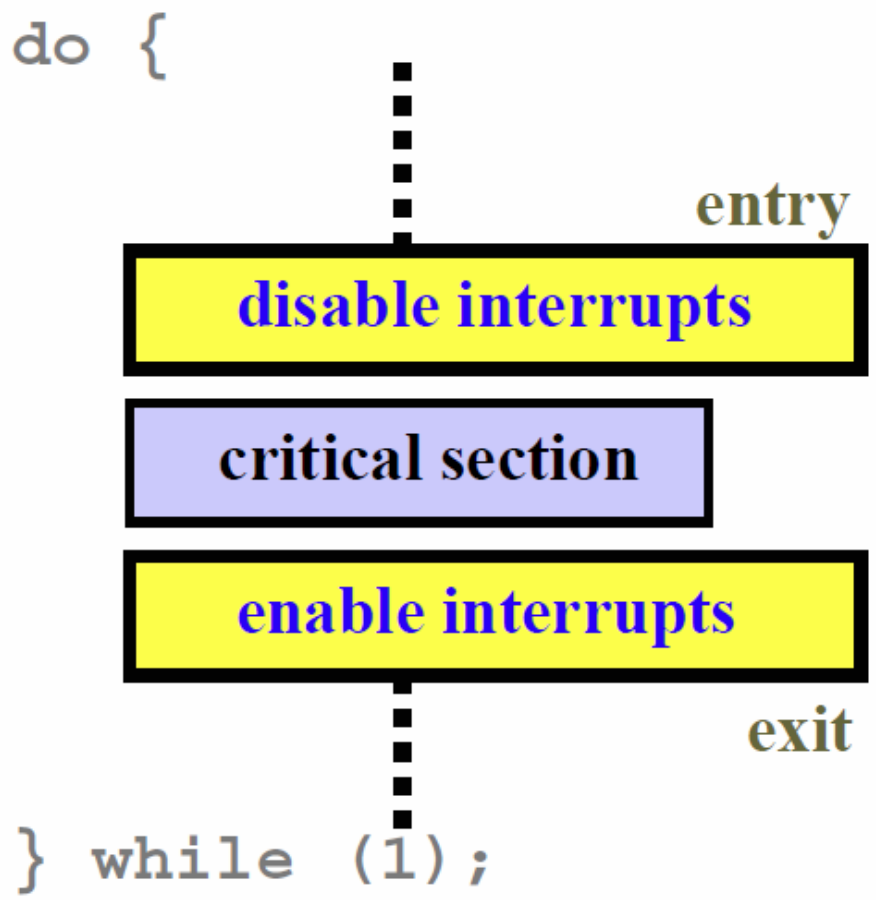
```
di;
saat = 0;
dakika = 0;
saniye = 0;
ei;
```

di : kesme devre dışı,
ei : kesme kullanılabilir.

Kesmeleri kapatmak/açmak (Disabling/Enabling interrupts)

Kesmeler devre dışı bırakıldığından, process kritik bölgede yürütülürken hiçbir bağlam anahtarlama (context switching) oluşmaz.

Fakat, kesmelere dayanan bazı özellikler (örneğin system clock) düzgün çalışmayabilir.



Kesmeleri kapatmak/açmak (Disabling/Enabling interrupts)

- **Ancak bu yöntemin de bazı sakıncaları vardır.**
- Bu yöntemin en önemli sakıncası, kesme düzeneğinin denetimini sistemde çalışan çok sayıda process'e, özellikle çekirdek katmanı dışındaki process'lere de dağıtmasıdır.
- Kesme düzeneğinin tutarlı yönetimi güvenli bir işletim için vazgeçilmez bir gerekliliktir.
- Sistemin kesmelere açılıp kapatılması gibi temel bir aracı her düzeyden (ayrıcalıklı ve sıradan) process'in insiyatifine bırakmak, kesmelerin yönetimini bu process'lerle paylaşmak anlamına gelir.

Kesmeleri kapatmak/açmak (Disabling/Enabling interrupts)

- Normal durumda process kesilene veya sistem çağrısı yapana kadar çalışmaya devam eder.
- Kritik bölge yürütümünde kesmeleri kapatarak karşılıklı dışlama sağlamış olur.
- Fakat kritik bölgelerin kesmelere kapalı olarak işletilmesi CPU'nun tüm dış uyarılara kapatılması anlamına gelir. Bu da sistemin genel başarımını olumsuz yönde etkiler.
- Bu yöntem işlemcinin birden fazla process'i zaman paylaşımlı olarak çalıştırma özelliğine müdahaledir.

Kesmeleri kapatmak/açmak (Disabling/Enabling interrupts)

- Bu yöntem tek işlemcili sistemlerde çalışır, çok işlemcili sistemlerde uygun çözüm değildir.
- İşlemciler bağımsız davranırlar. İşlemciler arasında kesme mekanizması yoktur.
- Çok işlemcili sistemlerde, kesmelerin disable/enable yapılması için tüm işlemcilere mesaj göndermek için zaman gereklidir ve sistem performansı düşer.
- Bu özelliği kullanan işletim sistemleri ölçeklenebilir değildir.

- <u>Yukarıda sayılan gerekçelere dayalı olarak kesme düzeneğinin kritik bölge probleminin çözümü için kullanılması, geçerli bir yol olarak benimsenemez.</u>

Özel Makine Komutlarına Dayalı Çözümler (Special Machine Instructions)

- Kritik bölge problemi için özel makine komutlarına dayalı çözümler, temel olarak **kilitleme (locking)** tabanlı yaklaşımlardır.

- Process kritik bölgeye girmeden önce **kilit kullanır**, kritik bölgeden çıkarken de **kilidi açılır**.

- Günümüzde modern işlemciler, kilidi uygulamak için kullanılabilecek özel atomik donanım komutları sağlar.
  - *Atomik= kesilmeyen (non-interruptable)*

Özel Makine Komutlarına Dayalı Çözümler

- ***Atomik (non-interruptible) komutlar :***
- *Tek bir komut çevriminde gerçekleşen komutlardır, kesilemezler.*
- *Böyle bir komut yürütüldüğünde, CPU'lar tarafından çeşitli aşamalarla yürütülen diğer tüm komutlar bu komut tamamlanıncaya kadar durdurulacaktır.*
- *Bu nedenle, iki farklı komut aynı anda verilirse, farklı CPU'larda olsalar da sırayla çalıştırılacaktır.*
- *Ayrıcalıklı (Privileged): Bu komutlar genel olarak ayrıcalıklıdır, yani sadece supervisor veya kernel modunda çalışabilirler.*

Özel Makine Komutlarına Dayalı Çözümler

- Çoğu modern bilgisayar sistemi, bir bellek hücresini test etmeyi/güncellemeyi veya iki bellek hücresini yer değiştirmeyi sağlayan özel donanım komutları sağlar.

- bir bellek hücresi içeriğini test edip değiştirme, (test and set lock -TS)

➢ test et ve değiştir komutu -TestAndSet()

- iki bellek hücresinin içeriğini yer değiştirme, (swap lock)
- yer değiştir komutu – Swap()

- bir bellek hücresinin içeriğini karşılaştırıp değiştirme, (compare and swap lock -CS)
- karşılaştır ve yer değiştir komutu - CompareAndSwap()

- Çok işlemcili bir ortamda da atomik olarak çalıştırılabilirler (her defasında bir CPU komutu yürütür: bellek erişimi içerir; bellek paylaşılır)

Özel Makine Komutlarına Dayalı Çözümler
Kilitleri Kullanarak Kritik Bölge Problemine Çözüm

- Kilit çözümünün genel düzeni şöyledir:

```
do {
        acquire lock
            critical section
        release lock
            remainder section
    } while (TRUE);
```

Kritik kod bölgesinin mümkün olduğunca küçük olması, yalnızca paylaşılan kaynaklara erişen/değiştiren bölümleri kilitlemesi gerekir.

Test and Set Komutunu Kilitleme için Kullanma

TestAndSet aşağıdaki fonksiyona eşdeğerdir:

```
bool TestAndSet(bool *target)
{
        bool rv = *target;
        *target = true;
        return rv;
}
```

Alternatif olarak;

```
bool TestAndSet(bool &target)
{
        bool rv = target;
        target = true;
        return rv;
}
```

Test and Set Komutunu Kilitleme için Kullanma

Shared data:

```
boolean lock = FALSE;
```

Process P_i

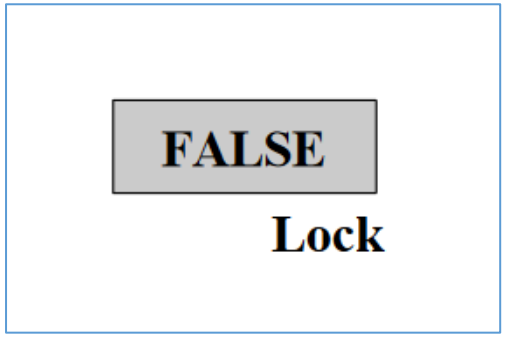
```
do {
    while (TestAndSet(&lock));
        critical section
    lock = FALSE;
        remainder section
} while (TRUE);
```

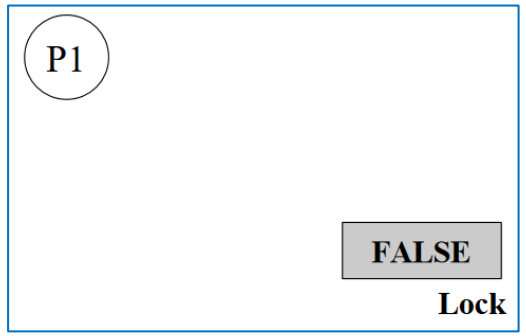
- lock: Boolean bir değişkendir ve başlangıçta FALSE olarak ayarlanır. Bu değişken, kritik bölgeye erişim için kullanılan kilidi temsil eder.
- Process, while(TestAndSet(&lock)) döngüsü içinde bekler.
- TestAndSet(&lock) komutu, lock değişkeninin değerini atomik olarak kontrol eder ve TRUE olarak ayarlar. Eğer lock değişkeni zaten TRUE ise, komut başarısız olur ve TRUE değerini döndürür.
- Eğer lock değişkeni FALSE ise, lock değişkeni TRUE olarak ayarlanır ve komut FALSE değerini döndürür. Bu, process'in kritik bölgeye girmesine izin verir.
- Process, kritik bölgeden çıktıktan sonra lock değişkenini FALSE olarak ayarlar.

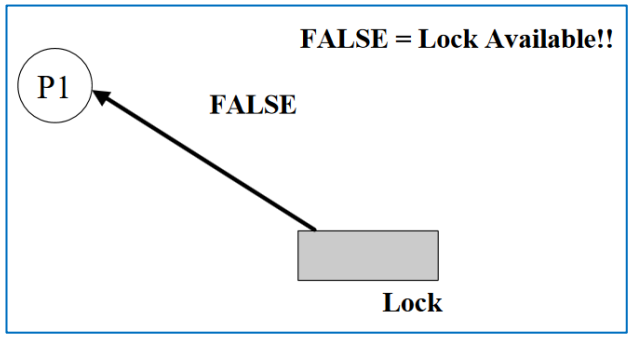
Test and Set Komutunu Kilitleme için Kullanma

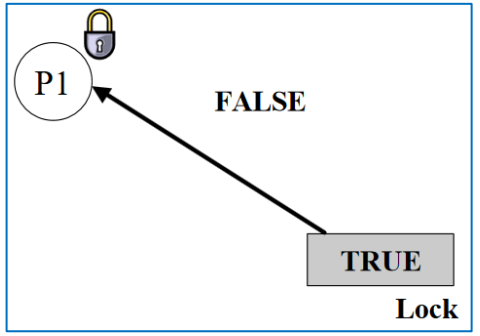
- Eğer lock değeri test edildiğinde TRUE ise, bir başka process zaten kilidi ele geçirmiştir (ve hala ona sahiptir).
- lock değişkenin bellekten okunup, alınması ve değerinin değiştirilmesi tek hamlede gerçekleştirilen bir işlemdir.
- Test and Set, komutunun işletimi sırasında, bu komutu işleten işlemcinin bellek erişim döngüsünün bölünmesine izin verilmez.
- Yani bu komut kesilemez.

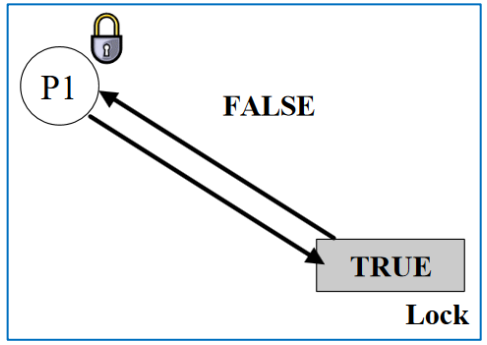
Test and Set Komutunu Kilitleme için Kullanma

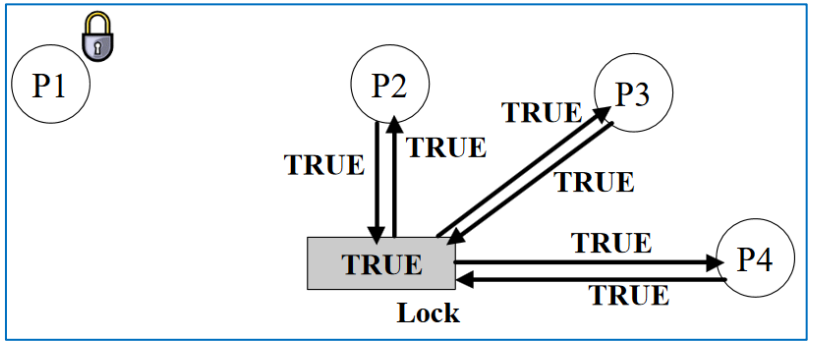


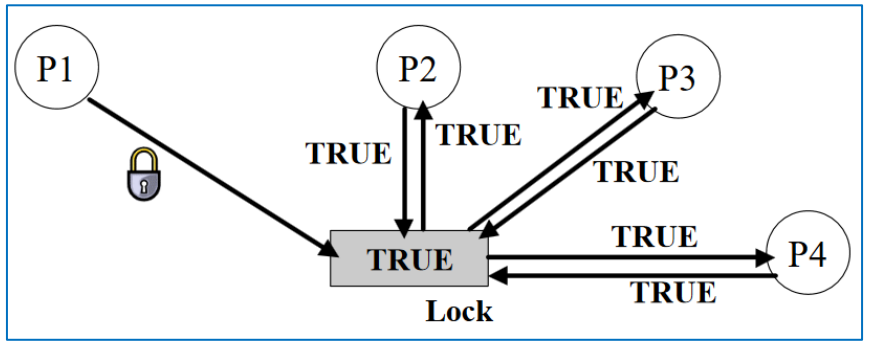




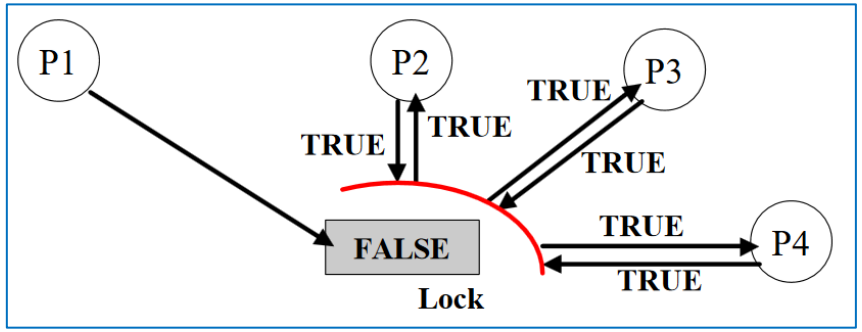


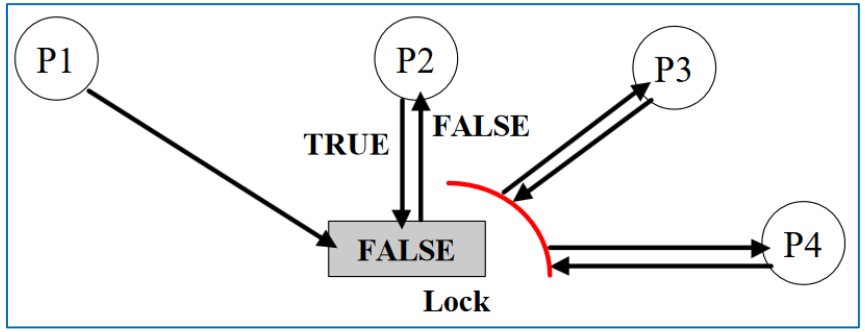


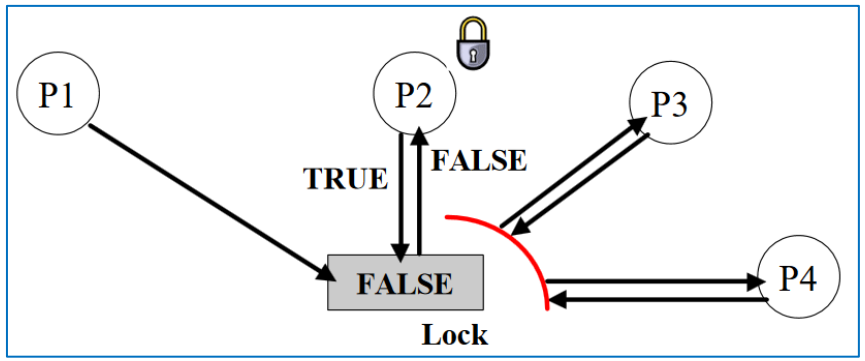


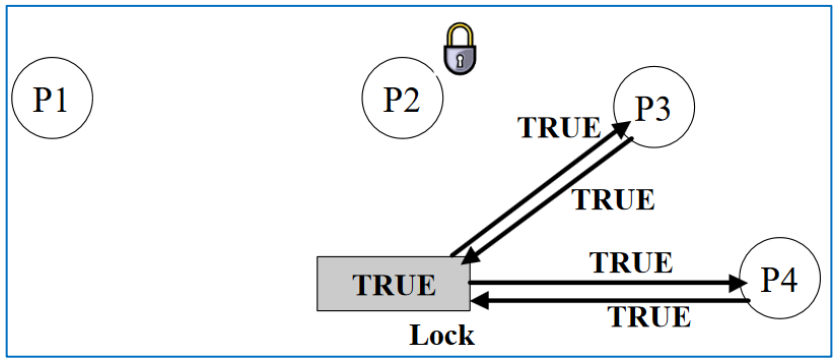


Test and Set Komutunu Kilitleme için Kullanma









Test and Set Komutunu Kilitleme için Kullanma

- Bir kilit sisteme iade edildiğinde ve diğer process'ler kilit üzerinde harekete geçmek için beklerken yalnızca bir process'in FALSE ile döneceğinden emin olunmalıdır.

- Bu yöntem ile karşılıklı dışlama (mutual exclusion) ve ilerleme (progress) sağlansa bile sınırlı bekleme (bounded waiting) gerçekleştirilemeyebilir.

- Test and Set işlemi, diğer senkronizasyon yöntemlerine göre daha yavaş olabilir ve sistem performansını etkileyebilir.

Test and Set Komutunu Kilitleme için Kullanma

// Kilidin ilk değeri sıfır olarak varsayılır
lock = 0

```
// Process 1

while true do

  // Kritik bölgeye girmeden önce kilidi ele geçir

  while TestAndSet(lock) == 1 do

    // Eğer kilid ele geçirilemezse, bu durum başka bir process'in kilit üzerinde olduğunu gösterir ve bu nedenle kritik bölgeye girene kadar beklenir.

  end while

  // Kritik bölgede yapılacak işlemler

…

  // Kritik bölgeden çıkarken kilidi serbest bırak

  lock = 0

  // Kritik bölge dışında yapılacak işlemler

  ...

end while
```

```
// Process 2

while true do

  // Kritik bölgeye girmeden önce kilidi ele geçir

  while TestAndSet(lock) == 1 do

    // Eğer kilid ele geçirilemezse, bu durum başka bir process'in kilit üzerinde olduğunu gösterir ve bu nedenle kritik bölgeye girene kadar beklenir.

  end while

  // Kritik bölgede yapılacak işlemler

  ...

  // Kritik bölgeden çıkarken kilidi serbest bırak

  lock = 0

  // Kritik bölge dışında yapılacak işlemler

  ...

end while
```

Swap Komutunu Kilitleme için Kullanma

- **Swap aşağıdaki fonksiyona eşdeğerdir: Atomik yapıdaki Swap iki değişkene sahiptir.**

```
void Swap (bool &a, bool &b)
{
    bool temp = a;
    a = b;
    b = temp;
}
```

- **Alternatif olarak,**

```
void Swap (bool *a, bool *b)
{
    bool temp = *a;
    *a = *b;
    *b = temp;
}
```

Swap Komutunu Kilitleme için Kullanma

- Shared data:
  boolean lock = FALSE;
- Process P_i

```
do {
  /* her process local Boolean key değişkenine
  sahiptir */
      key = TRUE;
      while (key == TRUE)
            Swap(&lock, &key);

            critical section
       lock = FALSE;
            remainder section
    } while (TRUE);
```

- Swap komutu da kesilemeyen özel donanım komutudur.
- lock ve key adı verilen iki değişkeni kullanır.
- İlk değeri FALSE olan global bir lock değişkeni ve her process için lokal bir key değişkeni tanımlanır.
- lock FALSE ise, bir process kritik bölgesine girebilir, aksi takdirde giremez.
- Her process'in kendine ait bir key değişkeni vardır. Bu değişken, process'in kritik bölgeye girmek isteyip istemediğini gösterir.
- Process, key değişkenini TRUE olarak ayarlayarak kritik bölgeye girmek istediğini belirtir.

Swap Komutunu Kilitleme için Kullanma

- Shared data:
  boolean lock = FALSE;
- Process P_i

```
do {
  /* her process local Boolean key değişkenine
  sahiptir */
      key = TRUE;
      while (key == TRUE)
            Swap(&lock, &key);

            critical section
       lock = FALSE;
            remainder section
    } while (TRUE);
```

- Process, while(key==TRUE) döngüsü içinde bekler. Bu döngü, key değişkeninin FALSE olarak ayarlanmasını bekler.
- Swap(&lock, &key) komutu, lock ve key değişkenlerinin değerlerini atomik olarak değiştirir.
- Eğer lock değişkeni FALSE ise, lock değişkeni TRUE olarak ayarlanır ve key değişkeni FALSE olarak ayarlanır. Bu, process'in kritik bölgeye girmesine izin verir.
- Eğer lock değişkeni TRUE ise, key değişkeni değişmeden kalır ve process while döngüsünde beklemeye devam eder.
- Process, kritik bölgeden çıktıktan sonra lock değişkenini FALSE olarak ayarlar.

Swap Komutunu Kilitleme için Kullanma

Swap işlemi Intel x86 mimarisinde XCHG makine komutu ile temsil edilir.

Assembly seviyesinde Swap makine komutunun kullanımı aşağıdaki gibidir.

```
enter_region:
        MOVE REGISTER,#1            | put a 1 in the register
        XCHG REGISTER,LOCK          | swap the contents of the register and lock variable
        CMP REGISTER,#0             | was lock zero?
        JNE enter_region            | if it was non zero, lock was set, so loop
        RET                         | return to caller; critical region entered

leave_region:
        MOVE LOCK,#0                | store a 0 in lock
        RET                         | return to caller
```

Swap Komutunu Kilitleme için Kullanma

- Swap komutu da, karşılıklı dışlama (mutual exclusion) ve ilerlemeyi (progress) sağlarken,
- yarış durumunun (race condition) önlenmesi için gerekli sınırlı bekleme (bounded waiting) şartını sağlamamaktadır.
- Swap mekanizması, process'lerin kritik bölgeye girmek için döngü içinde beklemesini gerektirir. Bu, process'lerin işlemci zamanını boşa harcamasına neden olabilir.

‘Compare and Swap’ Komutunu Kilitleme için Kullanma

- compare and swap komutu (CS), test and set (TS) komutuna benzerdir, ve donanımsal olarak atomiktir.

- CS komutu üç parametre alır: "bellek konumundaki belirli bir değer" (mem loc), "beklenen değer" (expected value) ve "yeni değer" (new value).

- Paylaşılan değişken: global bir lock değişkeni başlangıçta 0 yapılır.

Compare and Swap pseudo-code

```
CS (<mem loc>, <expected value>, <new value>)
  {
     if (<memloc> == <expected value>))
     {
       <mem loc> = <new value>;
       return 0;
     }
     else
     {
       return 1;
     }
```

compare and swap,
beklenen değer (expected value) ile, bir bellek konumundaki belirli bir değeri (mem loc) atomik olarak karşılaştırır
ve
yalnızca aynı oldukları zaman, bu bellek konumunun (mem loc) içeriğini bir yeni değere (new value) değiştirir.

‘Compare and Swap’ Komutunu Kilitleme için Kullanma

- Shared integer “lock” initialized to 0;
- Solution:

```
do {
 while (compare_and_swap(&lock, 0, 1) != 0)
            ; /* do nothing */
     /* critical section */
  lock = 0;
     /* remainder section */
  } while (true);
```

‘Compare and Swap’ Komutunu Kilitleme için Kullanma

- bellek konumunun içeriği (mem loc) ile karşılaştırılan beklenen değerin (expected value) aynı olması durumunda, bellek konumunun içeriği bir yeni değere (new value) değiştirilir.

- compare and swap metodunu çağıran process lock=1 yapar. Sonra process kritik bölümüne girebilir,

- Bu anda diğer process’ler kendi kritik bölgelerine giremez, çünkü lock beklenen 0 değerinde değildir.

- Bir process kendi kritik bölgesinden çıktığında lock değerini tekrar 0 yapar ve böylece başka bir process’in kritik bölgesine girebilmesini sağlar.

Özel Makine Komutlarının Dezavantajları

- Donanımsal özel makine komutları meşgul beklemeye neden olduğundan bekleyen process de işlemci zamanı harcar.

- Bir process kritik bölgesine girmeden önce kilidi sınamakta, kilidin meşgul olduğu durumlarda bu sınamaları yineleyerek kilidin açılmasını beklemektedir.
- *Bu durumda, erişeceği kilit meşgul olan bir process, sırası gelip CPU'ya anahtarlandığında kendisine ayrılan süre boyunca gerçekleşmeyecek bir koşulun oluşmasını bekleyerek, gereksiz yere CPU'yu meşgul edecektir.*

Özel Makine Komutlarının Dezavantajları

- Özel makine komutları kritik bölge sorununun çözümünde karşılıklı dışlama (mutual exclusion) ve ilerleme (progress) gereksinimlerini karşılarlar.
- Ancak kritik bölge sorunun sınırlı bekleme (bounded waiting) gereksinimini karşılamak için diğer mekanizmalarla tamamlanmaları gerekir.
- Bir process kritik bölgesinden çıktığında birden fazla bekleyen process varsa hangi process'in gireceği belirsizdir, sonsuz bekleme durumu yani açlık (starvation) oluşabilir.

- *Açlık (starvation) : Aynı kaynakları kullanan process'lerden bazılarının bekledikleri kaynaklara hiç erişememe durumu. Bekleyen process'ler sonsuz beklemeye girebilir.*